

SÜLEYMAN DEMİREL ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ
VERİ MADENCİLİĞİ FİNAL CEVAP ANAHTARI

ADI SOYADI:.....NO:.....

Sınav Yönergesi:

1. Sınav Süresi 30 dakikadır
2. Soruların puanları üzerinde belirtildiği gibidir.
3. Defter Kitap açıktır.
4. Cep telefonunun yanınızda veya yakınınızda bulunması kesinlikle yasaktır.

1. Bir çağrı merkezinde belirli bir saatte gelen çağrı sayıları aşağıdaki gibidir. Veriler, 20 saatlik gözlem sonucunda her saatlik dilimde kaç çağrı alındığını göstermektedir:

`calls_per_hour = [6, 2, 4, 7, 8, 3, 2, 5, 6, 4, 9, 3, 5, 4, 6, 2, 5, 7, 8, 5]`

Bu veriler ışığında :

- 1) Dağılımın tipini belirleyerek dağılım parametresini hesaplayınız
- 2) Tahmin edilen parametre değeri için bootstrap analizi yaparak dağılım parametresi için %95 güven aralığını (confidence interval) hesaplayınız.
- 3) Çağrı merkezi yöneticisi, "Saatlik ortalama çağrı sayısının 6'dan farklı olduğunu" iddia etmektedir. Bu iddiayı test etmek için bir **gerekli testi** uygulayınız.

Yukarıdaki işlemleri gerçekleştirmek için gerekli kod bloğu aşağıda verilmiştir. Gerekli boşlukları doldurunuz.

```
import numpy as np
from pingouin import ttest, anova, pairwise_tests, chi2_independence

np.random.seed(42)

calls_per_hour = [6, 2, 4, 7, 8, 3, 2, 5, 6, 4, 9, 3, 5, 4, 6, 2, 5, 7, 8, 5]

parameter_value = np.mean(calls_per_hour) (05)

print("Dağılım için parameter değeri:", parameter_value)

parameter_list = []

for i in range(1000): (05)
    samples = np.random.choice(calls_per_hour, size=len(calls_per_hour), replace=True) (05)
    parameter_list.append(np.mean(samples))

lower_bound, upper_bound = np.percentile(parameter_list, [2.5, 97.5]) (05)

print(f"Parametre için %95 güven aralığı: [{lower_bound:.2f}, {upper_bound:.2f}]")

results = ttest(x=calls_per_hour, y=6, alternative='two-sided') (05)
print(results) (05)
```

2. Bir bankanın kredi kararı vermesi için kullandığı "kredi.csv" isimli dosya okunmuş, dosyanın ilk 5 satırı ve dosya hakkında alınan bilgi sonucu aşağıda verilmiştir.

Veri setinin ilk satırları:

	KrediNotu	Yas	Maas	EvDurumu	KrediOnay
0	650.0	25.0	3000.0	Kira	Onaylandi
1	720.0	34.0	4200.0	EvSahibi	Onaylanmadi
2	610.0	45.0	3200.0	Kira	Onaylandi
3	750.0	NaN	5200.0	EvSahibi	Onaylandi
4	NaN	38.0	2000.0	None	Onaylanmadi

Data columns (total 5 columns):

#	Column	Non-Null Count	Dtype
0	KrediNotu	6 non-null	float64
1	Yas	6 non-null	float64
2	Maas	6 non-null	float64
3	EvDurumu	6 non-null	object
4	KrediOnay	6 non-null	object

Bu veri seti kullanılarak bir model kurularak başvuran kişiye Kredi verilir verilmeyeceği belirlenmek istenmektedir. Aşağıda bu işlemi gerçekleştirmek için gerekli kodlar aşağıda verilmiştir. Gerekli boşlukları doldurunuz.

```
import numpy as np
import pandas as pd
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, r2_score,
root_mean_squared_error
```

```
df = pd.read_csv("kredi_verisi.csv")
```

```
X = df.drop('KrediOnay', axis=1)
```

```
y = df['KrediOnay']
```

```
numeric_cols = list(X.select_dtypes(include='float64').columns)
```

```
categorical_cols = list(X.select_dtypes(include='object').columns)
```

```
num_imputer = SimpleImputer(strategy='mean')
```

```
X[numeric_cols] = num_imputer.fit_transform(X[numeric_cols])
```

```
cat_imputer = SimpleImputer(strategy='most_frequent', missing_values=None)
```

```
X[categorical_cols] = cat_imputer.fit_transform(X[categorical_cols])
```

```
X_encoded = pd.get_dummies(X, columns=categorical_cols, drop_first=True, dtype='int')
```

```
y = np.where(y=='Onaylandi', 1, 0)
```

```
model = LogisticRegression()
```

```
model.fit(X_encoded, y)
```

```
y_pred = model.predict(X_encoded)
```

```
# Buradaki boşluklarda f1 hesaplanacaktır
```

```
prec_score = precision_score(y, y_pred)
```

```
reca_score = recall_score(y, y_pred)
```

```
f1 = 2 * (prec_score * reca_score) / (prec_score + reca_score)
```